



UNIDADE II

Desenvolvimento Web com .NET

Prof. Dr. Tarcísio Peres

Por que persistir dados?

- Dados mantidos apenas em memória existem enquanto a aplicação está em execução.
- Ao encerrar ou reiniciar o servidor, listas e cadastros temporários são perdidos.
- Persistir significa gravar informações em um meio durável, como um banco de dados.
- Isso permite recuperar registros, manter histórico e dar continuidade às operações.
- Exemplo: um pet cadastrado no PetMatch deve continuar disponível na próxima sessão.
- A persistência transforma uma demonstração temporária em uma aplicação real.

Memória e banco de dados

- Memória é rápida, temporária e adequada ao processamento imediato.
- Ela simplifica protótipos, testes iniciais e exemplos sem dependências externas.
- Banco de dados mantém registros duráveis, estruturados e consultáveis.
- Também oferece relacionamentos, integridade e controle de transações.
- Consistência evita dados contraditórios, incompletos ou perdidos.
- Na prática, a memória apoia o fluxo; o banco preserva a fonte confiável dos dados.

ORM e Entity Framework Core

- No C#, os dados aparecem como classes, objetos, propriedades e coleções.
- No banco relacional, eles aparecem como tabelas, colunas, linhas e chaves.
- Um ORM faz o mapeamento entre esses dois modelos de representação.
- O EF Core consulta e salva dados por meio de entidades integradas à aplicação.
- Isso reduz conversões repetitivas e grande parte do SQL escrito manualmente.
- A abstração não elimina a necessidade de entender esquema, consultas e desempenho.

ADO.NET, Dapper e EF Core

- ADO.NET oferece controle direto de conexões, comandos, parâmetros e transações.
- É mais verboso, porém útil quando o cenário exige domínio fino sobre a execução.
- Dapper é um micro-ORM: mantém o SQL manual e simplifica o mapeamento para objetos.
- EF Core oferece uma abstração mais completa, com entidades e rastreamento de mudanças.
- A escolha equilibra produtividade, controle, legibilidade e desempenho.
- Uma aplicação pode combinar abordagens quando consultas específicas justificarem.

Entidade e ViewModel

- Entidade representa um conceito persistente com identidade no banco de dados.
- ViewModel representa os dados necessários para entrada ou saída de uma tela.
- No PetMatch, Pet é persistido; PetCreateViewModel recebe o formulário de cadastro.
- A entidade pode conter campos internos, chaves e tokens de concorrência.
- O *ViewModel* pode conter validações e campos adaptados à experiência da interface.
- Separar os dois evita expor detalhes do banco e misturar formulário com persistência.

DbContext: ponto central do EF Core

- O DbContext coordena a comunicação entre a aplicação e o banco de dados.
- Propriedades DbSet<T> expõem conjuntos de entidades, como DbSet<Pet>.
- Controllers MVC e endpoints de API podem receber o mesmo contexto por injeção.
- O mapeamento define chaves, campos obrigatórios, tamanhos e outras regras.
- O contexto rastreia alterações e as envia ao banco com SaveChangesAsync().
- Assim, páginas e APIs passam a compartilhar a mesma base persistente.

Interatividade

Em uma aplicação ASP.NET Core com EF Core, qual é a função principal do DbContext?

- a) Renderizar páginas Razor.
- b) Representar o ponto de acesso entre aplicação e banco de dados.
- c) Substituir o arquivo appsettings.json.
- d) Criar automaticamente controllers MVC.
- e) Executar apenas validações de formulário.

Resposta

Em uma aplicação ASP.NET Core com EF Core, qual é a função principal do DbContext?

- a) Renderizar páginas Razor.
- b) Representar o ponto de acesso entre aplicação e banco de dados.
- c) Substituir o arquivo appsettings.json.
- d) Criar automaticamente controllers MVC.
- e) Executar apenas validações de formulário.

SQLite e connection string

- SQLite armazena o banco em um arquivo local e não exige um servidor separado.
- A connection string informa onde está a base e quais opções serão utilizadas.
- Data Source=petmatch.db aponta para o arquivo físico do banco PetMatch.
- A configuração normalmente fica em ConnectionStrings no appsettings.json.
- O EF Core recebe essa configuração por `options.UseSqlite(connectionString)`.
- É uma escolha simples para estudo, demonstrações e aplicações locais.

Pooling de conexões e de contexto

- Abrir uma nova conexão completa para cada operação pode gerar custo desnecessário.
- Pooling=True permite ao provedor SQLite reaproveitar conexões disponíveis.
- AddDbContextPool reaproveita, de forma controlada, instâncias do DbContext.
- Pooling de conexão reutiliza o canal de comunicação com o banco.
- Pooling de contexto reutiliza o objeto do EF Core que organiza o trabalho.
- São níveis diferentes de otimização e devem manter operações curtas e isoladas.

Migrations e seed

- Uma migration registra uma mudança controlada no esquema do banco.
- Ela pode criar tabelas, adicionar colunas ou alterar regras de mapeamento.
- O snapshot guarda o estado atual do modelo para orientar novas comparações.
- `Database.Migrate()` aplica, na inicialização, as atualizações ainda pendentes.
- Seed insere dados iniciais quando a base está vazia.
- No PetMatch, o seed disponibiliza pets para teste sem duplicar registros existentes.

Transações protegem a consistência

- Uma transação reúne operações que devem concluir ou falhar como uma unidade.
- A regra é simples: tudo confirma ou tudo é desfeito.
- `SaveChangesAsync()` já usa transação quando o salvamento precisa de consistência.
- Fluxos com várias etapas podem usar `BeginTransactionAsync()` explicitamente.
- `CommitAsync()` confirma; `RollbackAsync()` desfaz quando ocorre uma falha.
- Isso impede que uma edição deixe o cadastro em um estado intermediário.

Concorrência otimista e RowVersion

- Dois usuários podem abrir o mesmo registro e tentar salvar mudanças diferentes.
- A concorrência otimista evita bloqueios longos e verifica o conflito ao gravar.
- A tela recebe um token RowVersion correspondente à versão originalmente lida.
- No salvamento, o EF Core compara o token original com o valor atual do banco.
- Se forem diferentes, ocorre `DbUpdateConcurrencyException` em vez de sobrescrita.
- No SQLite do exemplo, o token `byte[]` é renovado pela aplicação antes de salvar.

Consultas eficientes e base compartilhada

- `AsNoTracking()` evita rastreamento desnecessário em operações somente de leitura.
- Filtros devem ser aplicados antes de `ToListAsync()`, ainda na consulta ao banco.
- A projeção seleciona apenas os campos necessários para a resposta.
- DTOs como `PetResponse` impedem a exposição de detalhes internos, como `RowVersion`.
- Métodos assíncronos evitam bloquear o fluxo enquanto o banco responde.
- API e MVC podem consultar a mesma base e obedecer às mesmas regras de integridade.

Interatividade

Qual alternativa descreve corretamente o uso de `AsNoTracking()`?

- a) Impede que o banco salve qualquer alteração futura.
- b) Remove a necessidade de migrations.
- c) Evita rastreamento desnecessário em consultas somente de leitura.
- d) Cria uma transação explícita automaticamente.
- e) Converte uma entidade em ViewModel.

Resposta

Qual alternativa descreve corretamente o uso de `AsNoTracking()`?

- a) Impede que o banco salve qualquer alteração futura.
- b) Remove a necessidade de migrations.
- c) **Evita rastreamento desnecessário em consultas somente de leitura.**
- d) Cria uma transação explícita automaticamente.
- e) Converte uma entidade em ViewModel.

APIs REST

- Uma API REST expõe recursos para que outros clientes usem funções da aplicação.
- A comunicação acontece por requisições e respostas sobre o protocolo HTTP.
- Cada operação deve apresentar comportamento previsível para quem consome a API.
- Contratos estáveis reduzem suposições e evitam integrações frágeis.
- JSON é um formato comum para representar os dados enviados nas respostas.
- A mesma API pode atender a páginas web, aplicativos móveis e outros serviços.

Recursos e rotas

- Recurso é o conceito do domínio que a API disponibiliza por um endereço.
- GET /api/pets representa a coleção de pets acessível ao cliente.
- GET /api/pets/{id} representa um item específico identificado na rota.
- O método HTTP informa a operação; a rota indica sobre qual recurso ela atua.
- A resposta define o resultado esperado, inclusive códigos de sucesso ou falha.
- Método HTTP + rota + resposta formam o contrato de uma operação.

DTOs e contratos

- A entidade persistente pertence ao modelo interno da aplicação e do banco.
- Um DTO define a forma pública dos dados enviados ou recebidos pela API.
- PetResponse publica somente os campos necessários aos clientes do PetMatch.
- Dados internos, como o token de concorrência RowVersion, ficam ocultos.
- A projeção para DTO torna explícito o que pode sair da aplicação.
- Essa fronteira permite alterar a persistência sem quebrar o contrato público.

Filtros, ordenação e paginação

- Filtros reduzem a coleção por critérios como espécie, porte ou cidade.
- Os critérios chegam pela query string e devem possuir nomes previsíveis.
- Ordenação define a sequência dos resultados, por exemplo nome, cidade ou idade.
- Aceitar somente opções conhecidas mantém o comportamento estável.
- Paginação divide a coleção usando parâmetros como page e pageSize.
- Limitar pageSize impede respostas excessivamente grandes.

Resposta paginada

- items: contém somente os recursos pertencentes à página atual.
- page: informa o número da página retornada pela API.
- pageSize: informa a quantidade máxima de itens solicitada por página.
- totalItems: mostra quantos registros atendem aos filtros da consulta.
- totalPages: indica quantas páginas existem para o pageSize utilizado.
- O envelope permite ao cliente exibir resultados e construir a navegação.

Interatividade

Em uma API REST bem organizada, por que é recomendável usar DTOs nas respostas?

- a) Para substituir o banco de dados.
- b) Para expor diretamente todas as propriedades da entidade.
- c) Para controlar o formato público da resposta e ocultar detalhes internos.
- d) Para impedir o uso de paginação.
- e) Para remover a necessidade de rotas HTTP.

Resposta

Em uma API REST bem organizada, por que é recomendável usar DTOs nas respostas?

- a) Para substituir o banco de dados.
- b) Para expor diretamente todas as propriedades da entidade.
- c) Para controlar o formato público da resposta e ocultar detalhes internos.
- d) Para impedir o uso de paginação.
- e) Para remover a necessidade de rotas HTTP.

Consultas eficientes

- `AsNoTracking()` evita rastreamento quando a consulta é somente de leitura.
- A consulta deve permanecer como `IQueryable` enquanto os critérios são montados.
- Filtros com `Where()` precisam ser aplicados antes da materialização dos dados.
- A ordenação deve acontecer antes do recorte da página.
- `Skip()` ignora páginas anteriores e `Take()` limita a quantidade retornada.
- `Select()` projeta para o DTO antes de `ToListAsync()` executar a consulta.

Erros com Problem Details

- Erros também fazem parte do contrato público de uma API.
- Problem Details oferece um formato padronizado para representar falhas HTTP.
- title apresenta um resumo curto e legível do problema ocorrido.
- status informa o código HTTP associado, como 400 ou 404.
- detail explica a situação com informação útil para o cliente.
- Um pet inexistente pode retornar 404 sem criar uma resposta improvisada.

OpenAPI

- OpenAPI é uma especificação para descrever formalmente o contrato da API.
- O documento registra caminhos e métodos HTTP disponíveis.
- Também descreve parâmetros de rota, query string e corpo da requisição.
- Schemas definem a estrutura dos DTOs usados nas entradas e respostas.
- Códigos e tipos de resposta mostram o que cada operação pode produzir.
- O formato legível por ferramentas apoia documentação, integração e geração.

Swagger

- Swagger UI apresenta o contrato OpenAPI em uma interface visual no navegador.
- A tela organiza endpoints por rota e mostra os métodos HTTP disponíveis.
- Parâmetros, schemas e possíveis respostas podem ser consultados na interface.
- Try it out permite preencher dados e executar requisições durante os testes.
- No PetMatch, a interface fica disponível em /swagger no desenvolvimento.
- Ela deve permanecer restrita ao ambiente de desenvolvimento, fora das telas públicas.

NSwag e cliente TypeScript

- NSwag lê o documento OpenAPI produzido pela aplicação.
- A ferramenta gera tipos e métodos TypeScript correspondentes ao contrato real.
- nswag.json define o documento de entrada, o modelo do cliente e a saída.
- No PetMatch, o resultado é um PetMatchApiClient baseado em fetch.
- O build gera o OpenAPI e depois cria petmatch-api-client.ts.
- A geração reduz divergências de URLs, parâmetros e respostas entre front-end e API.

Interatividade

Qual é a principal diferença entre Swagger e NSwag no contexto do desenvolvimento web?

- a) Swagger cria o banco; NSwag cria migrations.
- b) Swagger permite visualizar e testar a API; NSwag gera cliente TypeScript a partir do contrato OpenAPI.
- c) Swagger substitui o controller; NSwag substitui o DbContext.
- d) Swagger é usado apenas em produção; NSwag apenas no navegador.
- e) Swagger serve para CSS; NSwag serve para HTML.

Resposta

Qual é a principal diferença entre Swagger e NSwag no contexto do desenvolvimento web?

- a) Swagger cria o banco; NSwag cria migrations.
- b) Swagger permite visualizar e testar a API; NSwag gera cliente TypeScript a partir do contrato OpenAPI.
- c) Swagger substitui o controller; NSwag substitui o DbContext.
- d) Swagger é usado apenas em produção; NSwag apenas no navegador.
- e) Swagger serve para CSS; NSwag serve para HTML.

ATÉ A PRÓXIMA!